講義 20a：SQL 聚合（Aggregate）與子查詢（Subquery）（授課順序：接在講義 13 之後；延續講義 20 的群組小計（Control Break））

> 對應練習：ex20a | 答案程式：`ZR_TR20A_SQL_AGG`

本講重點

- 為什麼「只要小計」時，不該把全部明細撈回內表
- 聚合函數：`COUNT( * )` / `SUM` / `AVG` / `MAX` / `MIN`
- `GROUP BY`：分組規則與最常見的語法錯誤
- `HAVING`：過濾「分組之後」的結果（跟 `WHERE` 的差別）
- `SELECT DISTINCT` 與 `COUNT( DISTINCT ... )`
- 子查詢：`IN ( SELECT ... )`、`EXISTS`、`NOT EXISTS`
- 什麼時候讓資料庫算、什麼時候撈回內表自己算

> 本講全程使用傳統寫法（先宣告結構與內表、`INTO CORRESPONDING FIELDS OF TABLE`），跟講義 6、11 一致。新式寫法（`@DATA` 行內宣告（Inline Declaration）等）留到講義 26 對照。

1. 為什麼要在資料庫算

講義 20 的營收報表，做法是把**全部**航班明細撈進內表，再用 `AT END OF` + `SUM` 算各公司小計。如果需求只是「每家公司一行統計」，明細其實用不到——大表撈幾十萬列回應用伺服器，只為了加總，白白浪費傳輸與記憶體。

Open SQL 讓資料庫自己彙總，**只把結果回傳**：

```
SELECT carrid SUM( seatsocc ) AS seats
  INTO CORRESPONDING FIELDS OF TABLE gt_agg
  FROM sflight
  GROUP BY carrid.
```

SFLIGHT 有上千列，回傳的只有「每家公司一列」。這就是講義 20 思考題 3 留的伏筆。

2. 聚合函數（Aggregate Functions）

函數	作用	備註
`COUNT( * )`	列數	最常用，不看欄位值
`COUNT( DISTINCT 欄位 )`	不重複值的個數	見第 5 節
`SUM( 欄位 )`	加總	只能用於數值欄位
`AVG( 欄位 )`	平均	目的欄位型別要留意（見下）
`MAX( 欄位 )` / `MIN( 欄位 )`	最大 / 最小值	數值、日期、文字欄位皆可

每個聚合都要用 `AS` 取別名，別名要跟目的結構的欄位名一致，`INTO CORRESPONDING FIELDS OF TABLE` 才對得上（講義 11 第 4 節同樣的規則）：

```
TYPES: BEGIN OF ty_agg,
         carrid    TYPE sflight-carrid,
         cnt       TYPE i,                          " 對應 COUNT( * ) AS cnt
         seats     TYPE i,                          " 對應 SUM( seatsocc ) AS seats
         avg_price TYPE p LENGTH 12 DECIMALS 2,     " 對應 AVG( price ) AS avg_price
       END OF ty_agg.
```

`AVG` 的目的型別要能放小數：實測 SFLIGHT 的 AA 公司平均票價是 422.94，接到不同型別的結果如下：

目的型別	結果
`TYPE i`	`422`（小數被截掉）
`TYPE f`	`4.2294E+02`（浮點數，可讀性差）
`TYPE p LENGTH 12 DECIMALS 2`	`422.94`（**建議**）

3. GROUP BY：分組

```
SELECT c~carrid c~carrname
       COUNT( * )         AS cnt
       SUM( f~seatsocc )  AS seats
       AVG( f~price )     AS avg_price
       MAX( f~price )     AS max_price
       MIN( f~price )     AS min_price
  INTO CORRESPONDING FIELDS OF TABLE gt_agg
  FROM sflight AS f
  INNER JOIN scarr AS c ON f~carrid = c~carrid
  GROUP BY c~carrid c~carrname
  ORDER BY c~carrid.
```

規則只有一條：SELECT 清單裡「不是聚合函數」的欄位，**都必須列在 `GROUP BY`**。上例的 `carrid`、`carrname` 是一般欄位，所以兩個都要寫進 `GROUP BY`。

漏寫的錯誤訊息長這樣：

```
The field "CONNID" from the SELECT list is missing in the GROUP BY clause.
```

這個限制的道理：分組之後「每組只剩一列」，一般欄位要有唯一的值才放得進去——`carrname` 跟 `carrid` 一對一，所以可以一起分組；`connid` 一家公司有很多條，沒辦法代表整組，就不能出現。

4. HAVING：過濾分組後的結果

子句	過濾的時間點	條件裡可以用
`WHERE`	分組**之前**（過濾明細列）	一般欄位
`HAVING`	分組**之後**（過濾整組）	聚合函數

```
* 已訂座位總數超過 10000 的公司
SELECT carrid COUNT( * ) AS cnt SUM( seatsocc ) AS seats
  INTO CORRESPONDING FIELDS OF TABLE gt_agg
  FROM sflight
  GROUP BY carrid
  HAVING SUM( seatsocc ) > 10000
  ORDER BY carrid.
```

想過濾「某家公司」用 `WHERE carrid = 'LH'`；想過濾「座位數大於某值的公司」只能用 `HAVING`，因為座位總數要分組之後才算得出來。能用 `WHERE` 擋掉的明細盡量用 `WHERE`——分組前先縮小資料量，比較快。

5. DISTINCT：不重複值

```
* 有航班起飛的機場（不重複）
SELECT DISTINCT airpfrom FROM spfli
  INTO TABLE gt_airp
  ORDER BY airpfrom.

* 不重複值的個數
SELECT COUNT( DISTINCT airpfrom ) FROM spfli INTO gv_dcnt.
```

實測 SPFLI 有 26 條航線，但只有 8 個不同的起飛機場——`COUNT( * )` 數的是列數（26），`COUNT( DISTINCT airpfrom )` 數的是不重複值（8）。

`SELECT DISTINCT 欄位` 跟 `GROUP BY 欄位`（不帶聚合）結果一樣；需要「順便算個數 / 加總」就用 `GROUP BY`，只要不重複清單用 `DISTINCT` 更直白。

6. 子查詢（Subquery）：SELECT 裡面再放一個 SELECT

括號裡的 SELECT 先跑，結果拿來當外層的條件。

`IN ( SELECT ... )`——有航線從 FRA 起飛的公司：

```
SELECT carrid FROM scarr
  INTO TABLE gt_carr
  WHERE carrid IN ( SELECT carrid FROM spfli WHERE airpfrom = 'FRA' )
  ORDER BY carrid.
```

`EXISTS`——同一個需求，內層引用外層的別名逐列比對：

```
SELECT carrid FROM scarr AS c
  INTO TABLE gt_carr
  WHERE EXISTS ( SELECT * FROM spfli
                   WHERE carrid = c~carrid AND airpfrom = 'FRA' )
  ORDER BY carrid.
```

`NOT EXISTS`——沒有任何航班的公司（這是講義 11「LEFT OUTER JOIN 找沒對到的列」的另一種寫法）：

```
SELECT carrid FROM scarr AS c
  INTO TABLE gt_carr
  WHERE NOT EXISTS ( SELECT * FROM sflight WHERE carrid = c~carrid )
  ORDER BY carrid.
```

跟 `FOR ALL ENTRIES`（講義 11 第 5 節）比較：

	子查詢	FOR ALL ENTRIES
資料庫往返	**1 次**（兩層 SELECT 一次送出）	2 次（先撈驅動內表）
空表陷阱	**沒有**	驅動內表是空的 → 全表撈回
條件來源	必須是資料庫裡的表	可以是程式裡加工過的內表

條件來源就是資料庫的另一張表時，優先用子查詢或 JOIN；只有條件來自程式內部算出來的內表，才需要 `FOR ALL ENTRIES`。

7. 傳統寫法的邊界

聚合函數裡**不能放算式**。講義 20 的營收是 `price * seatsocc`，想這樣寫：

```
SELECT carrid SUM( price * seatsocc ) AS revenue ...
```

編譯器會直接報錯：

```
The elements in the "SELECT LIST" list must be separated using commas.
```

原因是「聚合裡放算式」屬於新式 Open SQL 語法，一旦使用，整句必須改成新式寫法（欄位清單用逗號分隔、變數加 `@`）。傳統寫法下的做法有兩種：

- 只聚合單一欄位（如 `SUM( seatsocc )`、`SUM( paymentsum )`）
- 撈回明細內表，用講義 20 的 Control Break 算

新式寫法對照（7.40 之後）：聚合裡放算式

改成新式寫法（講義 6 §3.6），聚合函數裡就能放算式，營收直接在資料庫算好：

```
SELECT carrid,
       SUM( price * seatsocc ) AS revenue
```

```
  FROM sflight
  GROUP BY carrid
  ORDER BY carrid
  INTO TABLE @DATA(gt_rev).
```

本講其他寫法換成新式語法時的規則：

- `GROUP BY`、`ORDER BY` 的多個欄位用逗號：`GROUP BY c~carrid, c~carrname`。
- `INTO` 放在 `ORDER BY` 之後；`HAVING` 在 `GROUP BY` 之後、`ORDER BY` 之前。
- 一次接進多個變數：`INTO ( @gv_cnt, @gv_sum )`，括號裡每個變數都要加 `@`。
- 子查詢的寫法不變，只有外層 `SELECT` 要照新式規則。

練習 20a 的新式寫法版本見答案程式 `ZR_TR20A_SQL_AGG_NEW`（輸出跟原版逐行比對相同），第 5 段「內表累加」順便改用 `REDUCE ... FOR d IN gt_det WHERE ( carrid = ... )`，只加符合條件的列。

7.1 進階補充：CTE（通用表運算式，Common Table Expression）、`FROM @itab`、視窗函數（Window Function）

下面三個功能都要用新式寫法，本系統 7.54 都能用（2026-10-08 用驗證程式 `ZR_TR06_OSQL_CHK` 在本系統 7.54 實測）。實務上比較少見，讀到時看得懂即可。

CTE（`WITH`）：先算出一個暫存結果，再拿來查

```
WITH +lh AS ( SELECT carrid, connid, seatsocc FROM sflight WHERE carrid = 'LH' )
SELECT connid, SUM( seatsocc ) AS seats
  FROM +lh
  GROUP BY connid
  INTO TABLE @DATA(gt_seats).
```

`+lh` 是暫存結果的名稱（一定要 `+` 開頭），後面的 SELECT 把它當成一張表來用。適合「先篩選、再彙總」分好幾步的查詢，比層層巢狀的子查詢好讀。

`SELECT ... FROM @itab`：把內表當成資料來源（Internal Table as Data Source）

```
SELECT carrid, connid FROM @gt_keys AS k
  WHERE connid > '0400'
  INTO TABLE @DATA(gt_sel).
```

用 SQL 的 `WHERE`、`GROUP BY` 處理一張已經在記憶體裡的內表。內表名稱前面要加 `@`，並給一個別名（`AS k`）。

視窗函數（Window Function）`OVER( )`：不用 `GROUP BY` 也能在每一列旁邊放小計

```
SELECT carrid, connid, fldate, seatsocc,
       SUM( seatsocc ) OVER( PARTITION BY carrid ) AS carr_total
  FROM sflight
  INTO TABLE @DATA(gt_detail).
```

`GROUP BY` 會把同一組壓成一列；`OVER( PARTITION BY carrid )`（`PARTITION BY` 叫分區，Partition）則是每一列保留明細，旁邊多一欄「這家公司的合計」。實測 LH 每一列的 `carr_total` 都是 17,114，跟講義 20 的 Control Break 小計相同。

8. 選擇準則：資料庫算 vs 內表算

情境	建議
只要統計結果、不要明細	`GROUP BY` + **聚合**（資料庫算）
要明細，又要每組小計 / 總計	撈明細進內表，用 Control Break（講義 20）
算式很複雜（多欄位運算、要用 ABAP 函數）	撈回內表，用 ABAP 算
只要判斷「有沒有符合的資料」	`EXISTS` 子查詢，或 `SELECT SINGLE`

大原則：**能讓資料庫算完再回傳的，就不要搬資料回來算**。

9. 常見錯誤與陷阱

症狀	原因
`The field "XXX" from the SELECT list is missing in the GROUP BY clause`	一般欄位沒列進 `GROUP BY`
聚合結果那欄是空的	沒用 `AS` 取別名，或別名跟結構欄位名不一致
`AVG` 的結果沒有小數	目的欄位是 `i`，改用 `p ... DECIMALS 2`
`SUM( a * b )` 報「must be separated using commas」	聚合裡放算式屬於新式語法（見第 7 節）
查不到資料，`sy-subrc` 卻是 0	沒有 `GROUP BY` 的聚合一定回傳一列（COUNT 是 0、SUM 是 0）——判斷有沒有資料要看 `COUNT( * )`
想用聚合條件過濾，`WHERE SUM(...) > 100` 報錯	聚合條件要放 `HAVING`，`WHERE` 只能用一般欄位

其中「`sy-subrc` 是 0 但其實沒資料」最容易被忽略。實測對不存在的公司 `ZZ` 執行 `SELECT COUNT( * ) SUM( seatsocc ) ... WHERE carrid = 'ZZ'`，結果 `sy-subrc = 0`、`sy-dbcnt = 1`、`COUNT = 0`。一般 `SELECT` 沒資料 `sy-subrc = 4`，聚合則不是，這個差別要記住。

10. 課堂練習

完成 ex20a：用 `GROUP BY` 統計各航空公司航班數、座位、票價；加 `HAVING`；用 `DISTINCT` 與子查詢（`IN` / `EXISTS` / `NOT EXISTS`）回答三個問題；最後跟內表累加的結果交叉驗證。